

A
Project Synopsis
On
Buyer – Vendor Ticket Management
System (Web Application)

Submitted to



by
Omkar Amit Gore

Document Title: Buyer – Vendor Ticket Management System (Web Application)

Project Group: Group 5

Prepared By: Omkar Amit Gore

Branch: Information Technology

Approved By: Mr. Vijay Venu

Designation: Solutions Analyst

Submission Date: 16-03-2026

1. Problem Statement:

In many organizations, communication between buyers (internal teams or customers) and vendors (service providers or suppliers) is often managed through emails, phone calls, or informal communication channels. This approach leads to several operational challenges such as lack of proper tracking, delayed responses, poor visibility of issue status, and difficulty in maintaining historical records of issues and resolutions.

Without a structured system, it becomes difficult for both buyers and vendors to monitor the lifecycle of requests, prioritize critical issues, assign responsibilities, and ensure timely resolution. Additionally, management teams face challenges in

analyzing recurring issues, monitoring vendor performance, and improving overall service efficiency.

To address these challenges, there is a need for a centralized **Buyer–Vendor Ticket Management System** that enables structured communication, transparent issue tracking, and efficient resolution management through a web-based platform.

The proposed system will allow buyers to raise tickets for issues or requests related to products or services, while vendors will be able to view, manage, and resolve those tickets within the system. The platform will also provide tracking, status updates, and historical data for analysis and reporting.

This project will help freshers understand how enterprise systems manage workflows, user roles, and issue tracking processes in real-world scenarios.

2. Existing System & Limitations:

In the traditional service and support industry, communication between buyers (customers) and vendors (service providers) is largely handled through manual or legacy processes, leading to significant delays and organizational challenges for all parties.

1. Ticket Submission

Currently, buyers are required to report issues either by calling a helpdesk, sending unstructured emails, or visiting a vendor in person. This approach often leads to **lost requests**, as manual double-checking of incoming messages is time-consuming and prone to human error. Additionally, buyers face significant friction in explaining their problems multiple times to different people, resulting in dissatisfaction.

2. Information Management

Service and support records are primarily maintained in physical logbooks or simple, disconnected spreadsheets. This lack of centralization makes it nearly impossible to access historical ticket information quickly. Data is vulnerable to being misplaced, and generating summaries or monthly performance reports is labour-intensive and inaccurate, reducing overall operational efficiency.

3. Transparency & Tracking

Buyers currently have very limited visibility into the resolution process. They must repeatedly call or message the vendor to know if their ticket has even been seen. This lack of **real-time status updates** creates confusion and lowers trust, as buyers are unable to track progress or see which specific vendor is working on their issue.

4. Assignment & Workflow

Tickets are assigned to vendors manually, often based on verbal instructions or memory rather than availability and expertise. This leads to an **uneven workload distribution**, where some vendors are overwhelmed while others are idle. Manual tracking increases the likelihood of tickets being forgotten or delayed, further affecting the quality of service.

Summary of Limitations

The current manual system is highly inefficient both operationally and administratively. Manual bottlenecks create poor data management and a total lack of transparency, which negatively impacts the buyer's experience. These challenges highlight the critical need for a **digital full-stack solution** that can automate and streamline ticket creation, vendor assignment, real-time tracking, and historical record management.

The proposed Ticket Buyer-Vendor System aims to address these gaps by providing a centralized, web-based platform that improves communication efficiency, data transparency, and overall satisfaction for both buyers and vendors.

3. Objective:

The primary objective of this project is to design and develop a **web-based Buyer– Vendor Ticket Management System** that enables efficient issue reporting, tracking, and resolution between buyers and vendors.

Key objectives include:

1. Develop a centralized platform where buyers can create and track service or support tickets.
2. Enable vendors to view, update, and resolve assigned tickets.
3. Implement a structured ticket lifecycle including ticket creation, assignment, progress updates, and closure.
4. Provide visibility into ticket status, priority levels, and resolution history.
5. Ensure proper role-based access control for buyers, vendors, and administrators.
6. Maintain a historical record of all tickets for monitoring and reporting purposes.
7. Improve communication efficiency and accountability between buyers and vendors.

4.Scope of the Project:

The proposed Ticket Buyer-Vendor System is a full-stack digital solution designed to bridge the communication gap between service seekers and providers.

Core Project Capabilities

- 1. **Serve three distinct user roles: Buyer** (Customer), **Vendor** (Service Provider), and **Admin** (System Manager).
- 2. **Include professional web-based interfaces** for all user roles, built using **React.js** and styled with **Tailwind CSS v4** for a responsive, modern experience.
- 3. **Utilize a RESTful backend API** developed with the **Django REST Framework (DRF)** to manage business logic and data security.
- 4. **Connect to a PostgreSQL database**, ensuring reliable relational data storage for tickets, users, and historical records.
- 5. **Ensure seamless data flow** between the React frontend, Django API backend, and PostgreSQL database layers.
- 6. **Incorporate Role-Based Access Control (RBAC)** to ensure users only access data relevant to their specific role.
- 7. **Be visualized and prototyped** using **Figma** to ensure high-quality UI/UX design standards are met before development.

Tier	Technology	Responsibility
Client Tier	React.js + Tailwind CSS	User interaction, UI state management, and API consumption.
Application Tier	Django (Python)	Business logic, authentication, and RESTful API endpoints.
Data Tier	PostgreSQL	Persistent storage of users, tickets, and categories.

5. Proposed System Overview:

The **Ticket Buyer-Vendor System** is designed to bring buyers, vendors, and administrators onto a single, centralized digital platform. The system aims to improve communication efficiency, enhance overall service performance, and enable organizations to derive actionable insights from ticket-related data.

By digitizing and automating support workflows, the system reduces manual intervention, improves transparency, and ensures seamless coordination among all stakeholders involved in the service request process.

Key Objectives and Functionality

1. **Buyer Empowerment:** The system allows buyers to self-register, submit service tickets online, receive real-time status updates, and access a complete digital history of their past requests. This enhances convenience, transparency, and trust.
2. **Vendor Efficiency:** The platform provides vendors with digital tools to efficiently manage assigned tasks, update ticket progress, and maintain records of resolved issues. This results in faster service delivery and reduced operational errors.
3. **Real-Time Interaction:** The system offers instant status notifications while ticket resolution is in progress, ensuring continuous communication and significantly enhancing user satisfaction.

6. Proposed System Architecture / Workflow

The Ticket Buyer-Vendor System follows a three-tier architecture consisting of the **Client Tier** (Frontend), **Application Tier** (Backend), and **Data Tier** (Database). This architectural approach ensures scalability, security, modularity, and ease of maintenance by clearly separating user interaction from business logic and data storage.

6.1 Architecture Components

1. Client Tier – Frontend (React + Vite)

The Client Tier represents the user-facing layer of the system. It is implemented as a web-based application using **React.js** and **Vite**, providing a responsive and modern user interface that works seamlessly across desktop and mobile browsers.

This layer is responsible for:

1. **Capturing user input** (e.g., ticket details, login credentials).
2. **Displaying data** received from backend APIs (e.g., ticket lists, status updates).
3. **Managing UI state and navigation** using React hooks and routing.

Key Components:

4. **UI Components:** Buttons, forms, tables, cards, and reusable Tailwind-styled elements.
5. **Pages & Views:** Login, Buyer Dashboard, Vendor Dashboard, and Admin management views.
6. **Frontend Services:** API services (Axios) for authentication, ticket submission, and data fetching.

2. Application Tier – Backend (Django + REST Framework)

The Application Tier forms the core of the system, handling business logic, security, and data processing. It is built using **Django** and the **Django REST Framework (DRF)**, following a RESTful API architecture.

Major Backend Components:

1. **Middleware Layer:** Handles authentication (JWT), authorization, request validation, CORS handling, and error management.
2. **Routes/URL Layer:** Exposes REST APIs for system functionalities such as:
 1. Authentication & User Profiles.
 2. Ticket Creation & Management.
 3. Vendor Assignments.
 4. Category & Status Management.
3. **Views/Controllers Layer:** Implements business logic such as Role-Based Access Control (RBAC), ticket workflow handling, data validation, and processing requests received from the frontend.

3. Data Tier – Database (PostgreSQL)

The Data Tier is responsible for persistent data storage and management. The system uses **PostgreSQL**, a reliable relational database that supports structured data relationships and transactional consistency.

Database Structure:

- 1 **Core Tables:** Users (Buyers, Vendors, Admins), Categories.
- 2 **Transaction Tables:** Tickets, Assignments, Resolution Logs, Comments.
- 3 **Utility Tables:** JWT Tokens, Database Migrations.

6.2 Workflow Summary

The following workflow explains how data flows through the Ticket Buyer-Vendor System across all tiers to resolve a user issue:

1. Ticket Submission Process

Buyers log in through the React frontend and submit a service ticket by providing a title, description, and category. This request is sent via **REST APIs** to the Django backend and persisted in the **PostgreSQL** database.

2. Review & Vendor Assignment

The **Admin** reviews incoming tickets through the central dashboard to verify the issue. Once validated, the Admin assigns the ticket to a specific **Vendor** based on the category or expertise, and the system updates the ticket status to "Assigned".

3. Ticket Execution & Resolution

Vendors access their assigned tickets via their specific frontend dashboard. They update the job status (e.g., from "Open" to "In Progress"), record their resolution steps, and add any necessary comments. The backend synchronizes these updates in the database in real-time.

4. Completion & Verification

After the vendor marks the ticket as "Completed," the system logs the final resolution time. The **Buyer** can then view the resolved ticket through their frontend and, if necessary, provide feedback or close the ticket officially.

5. Tracking & Audit History

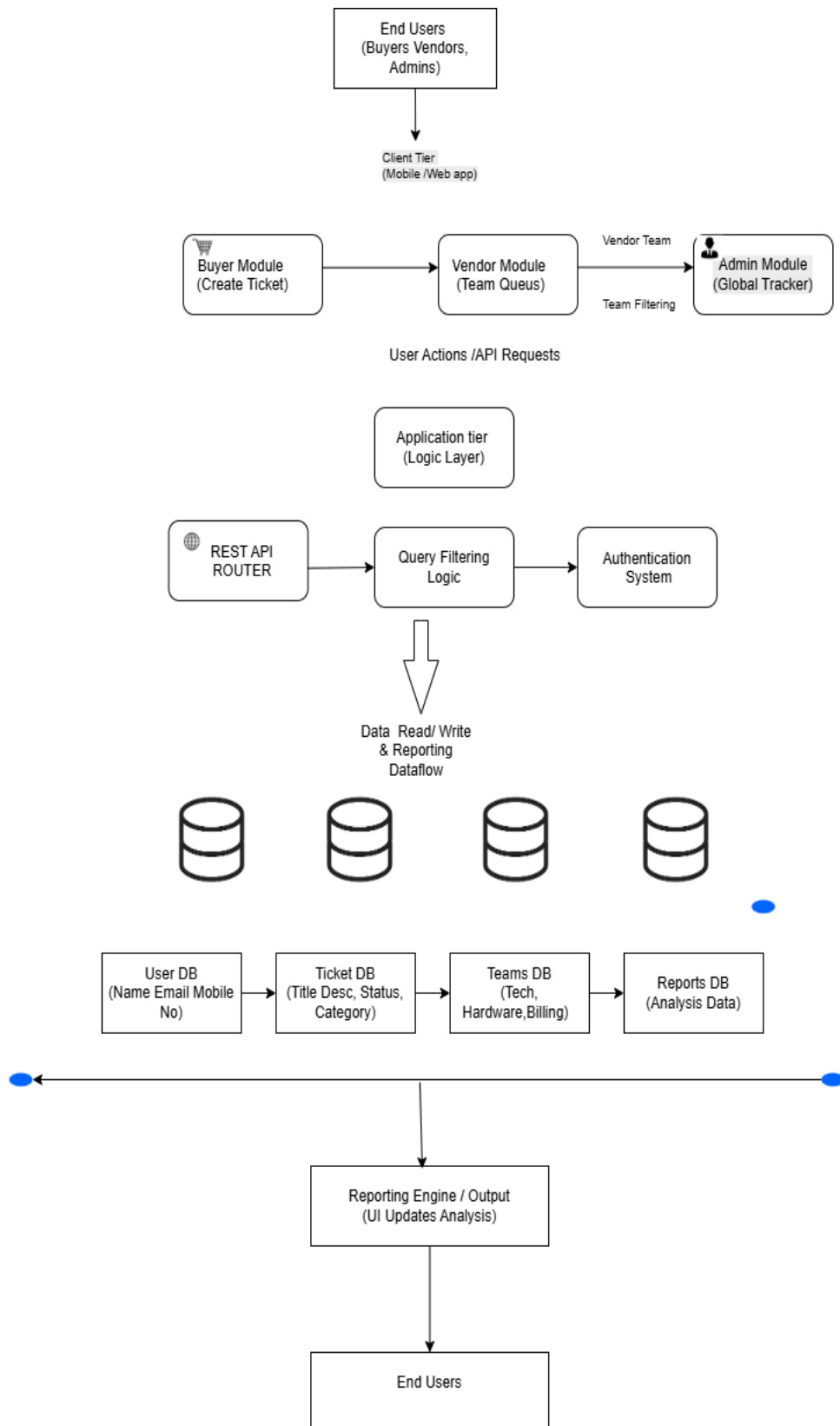
Both buyers and administrators can view **real-time status updates**, ticket progress, and a complete history of all past interactions. All actions are logged and stored in PostgreSQL for transparency, accountability, and future reference.

6.3 Architectural Benefits

Implementing this three-tier architecture provides several key advantages for the long-term health of the project:

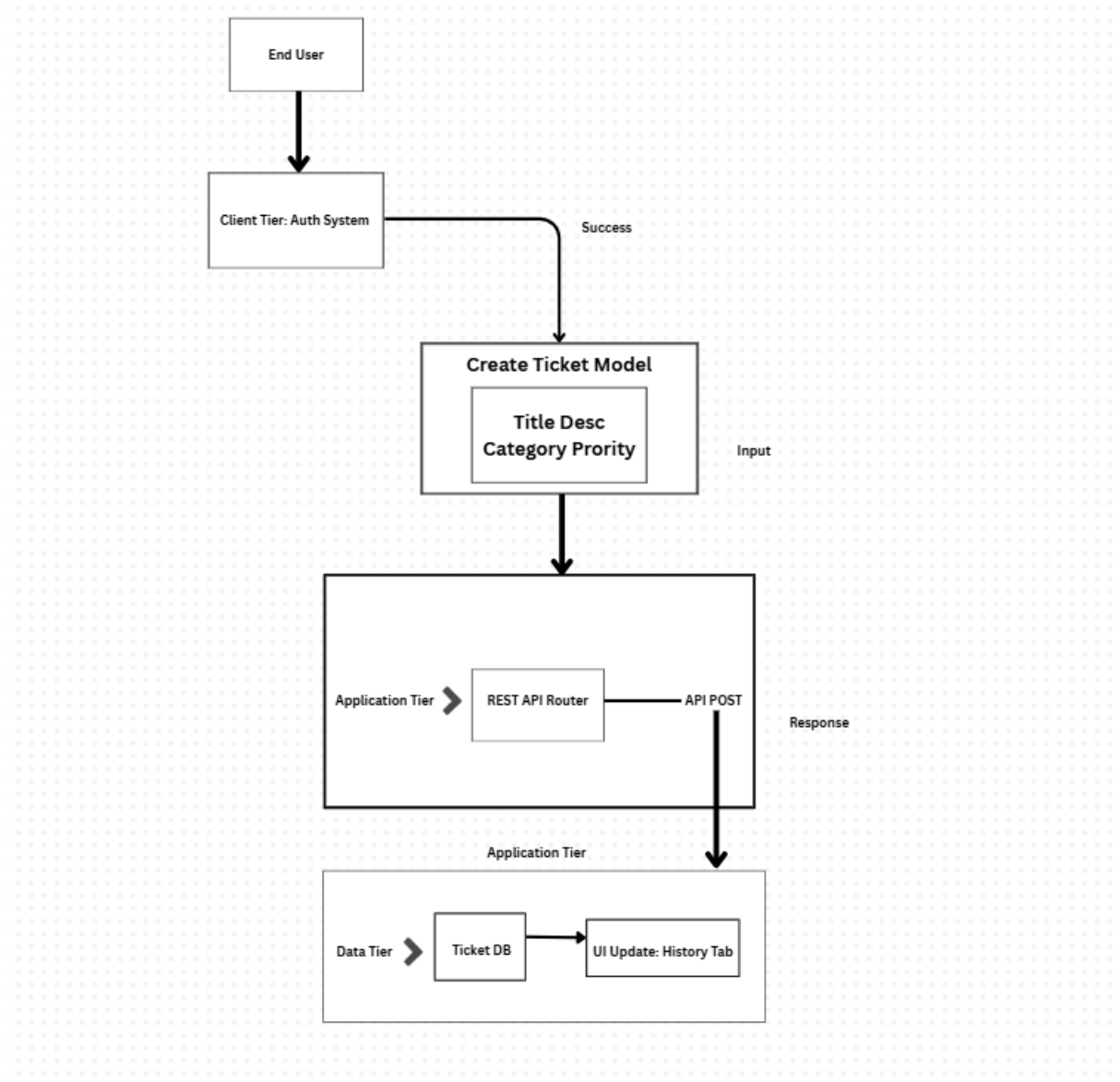
- 1 **Separation of Concerns:** Ensures a clear division between the user interface (React), business logic (Django), and data storage (PostgreSQL).
- 2 **Scalability:** Allows each tier to be scaled independently as the number of buyers or vendors grows.
- 3 **Security:** Protects sensitive user data through **JWT authentication** and strict **role-based access control (RBAC)**.
- 4 **Maintainability:** Utilizes modular components in both the frontend and backend to simplify future upgrades and feature additions.
- 5 **Performance:** Leverages optimized API communication and proper database indexing to ensure fast response times for dashboard metrics.

6.4 Architectural diagram



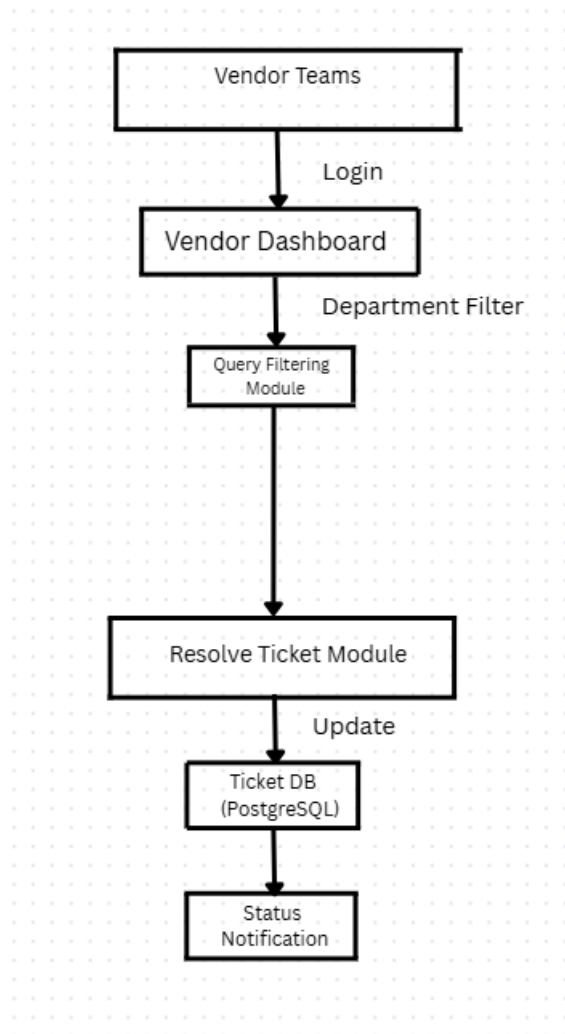
7. Modules Description

1. Buyer Module



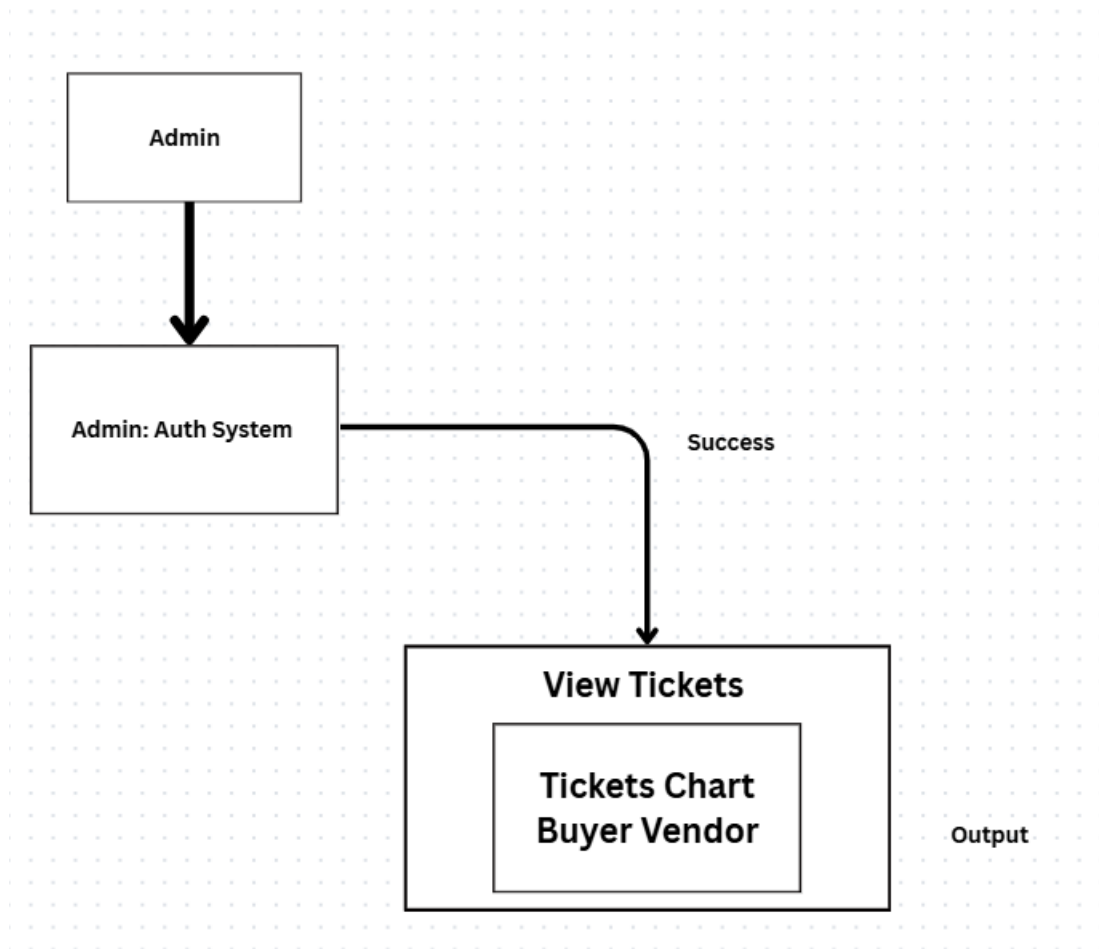
The Buyer Module of the TicketHub Pro system is designed as a high-fidelity, streamlined interface for end-users to report technical issues and track their resolution progress. Following the recent massive typography and layout updates, the module emphasizes clarity, security, and enterprise-grade aesthetics.

2. Vendor Module



The Vendor Module (Expert Interface) of TicketHub Pro serves as the "Resolution Engine" of the system. It is designed for technical experts and department heads to manage, analyze, and resolve incoming technical records routed from the Buyer Module.

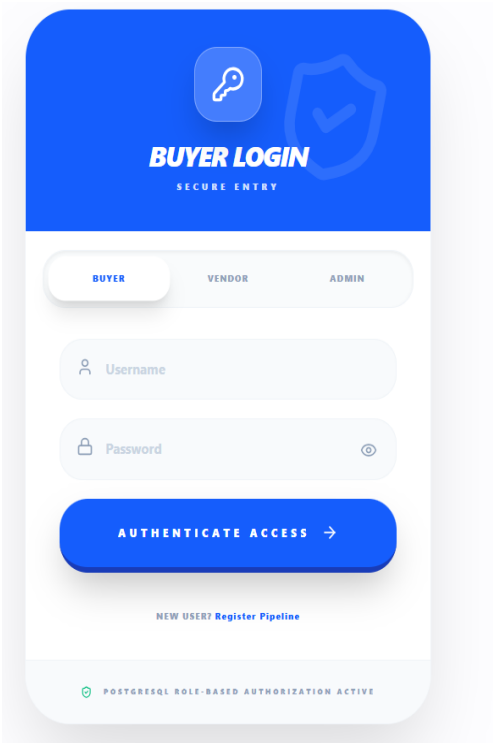
3. Admin Module



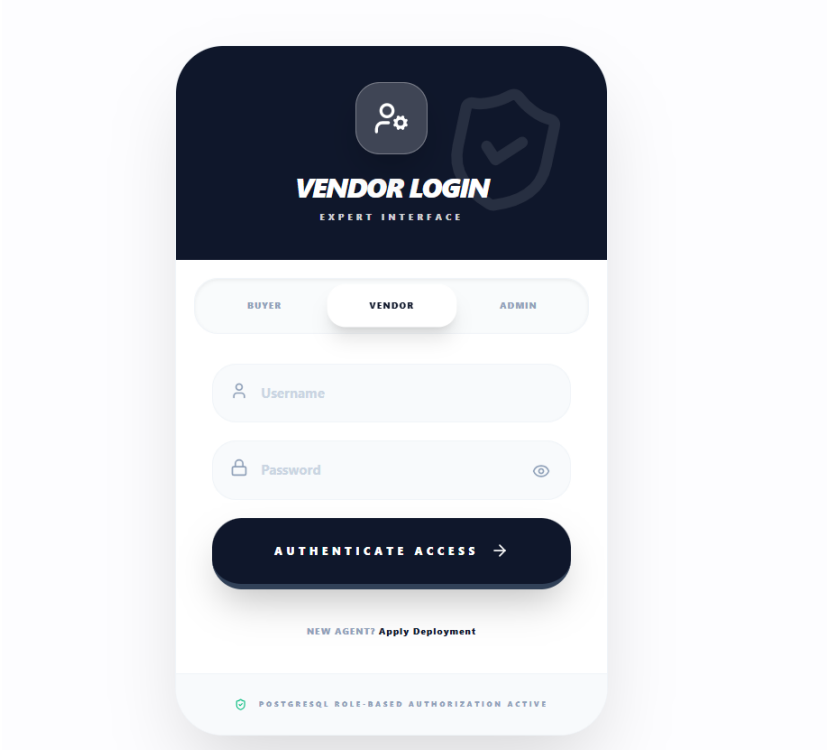
The Admin Module (Full Oversight Engine) is the highest tier of the TicketHub Pro ecosystem. It is designed for system administrators and managers who require a bird's-eye view of the entire enterprise support pipeline.

Unlike the Buyer or Vendor modules, which are restricted to personal or departmental data, the Admin Module has global read/write permissions across all technical records and user profiles.

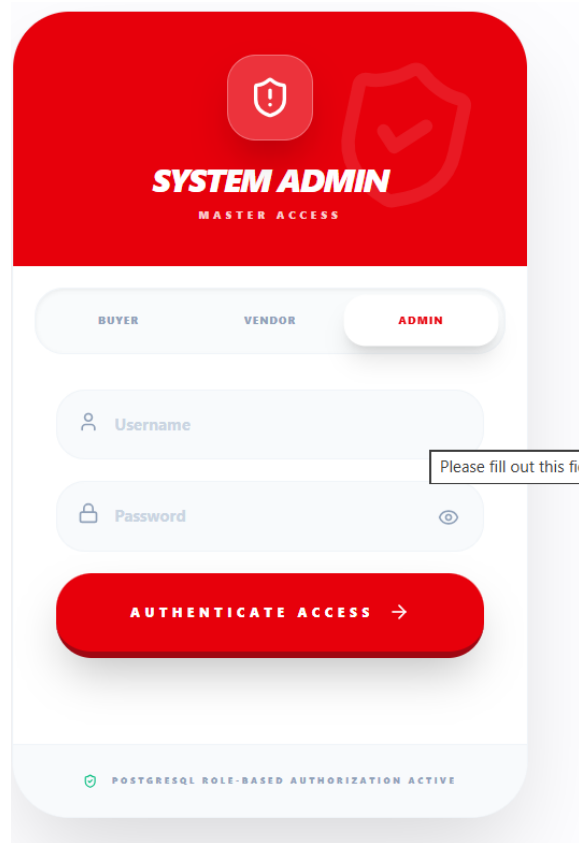
8. Output of Project



Buyer Login System

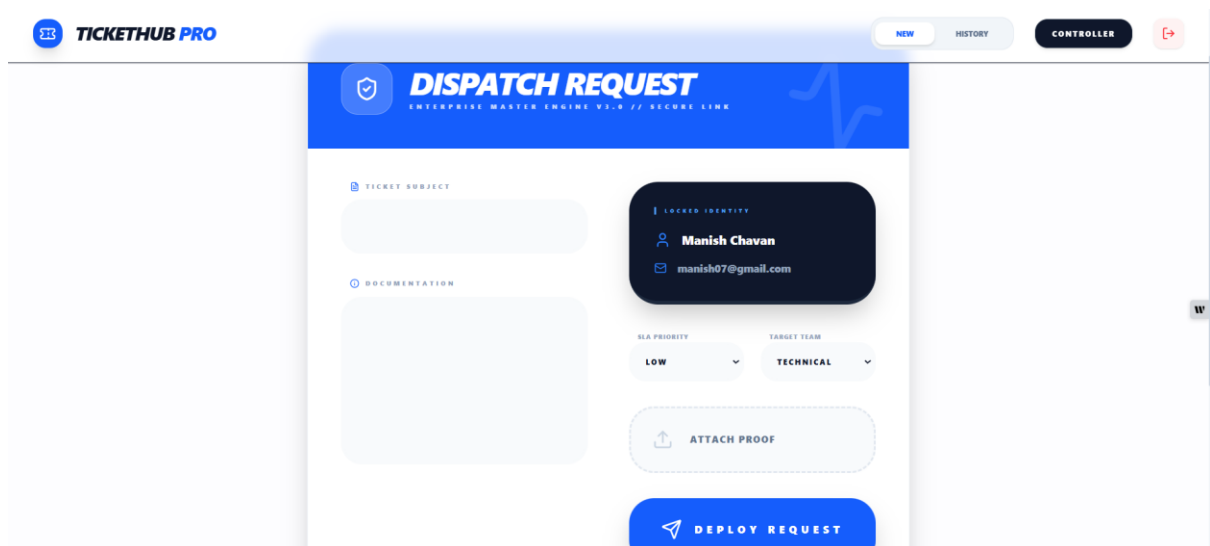


Vendor Login System



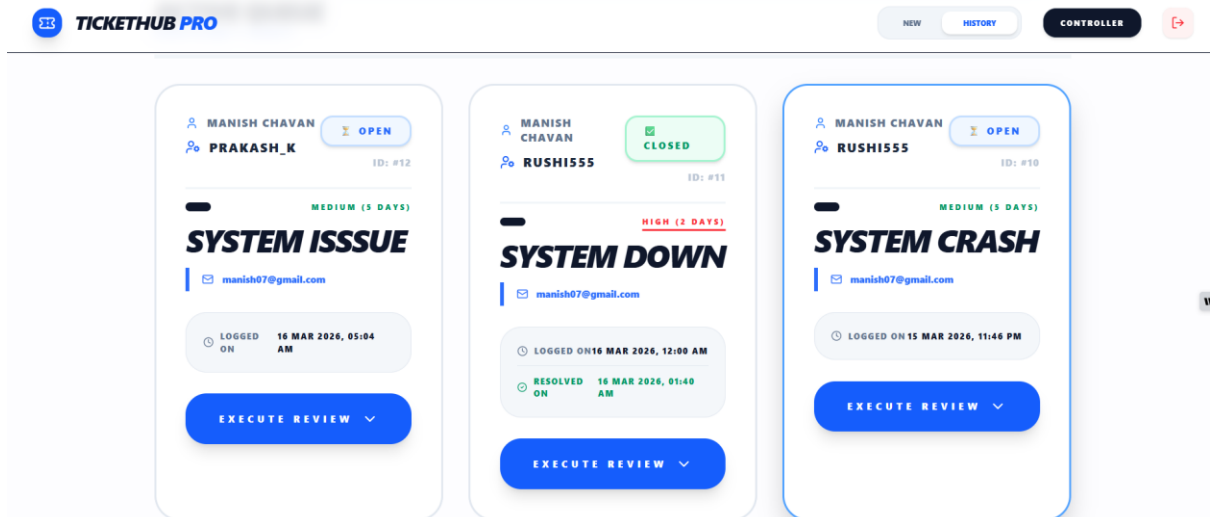
The Admin Login System UI features a red header with a shield icon and the text "SYSTEM ADMIN MASTER ACCESS". Below the header, there are three tabs: "BUYER", "VENDOR", and "ADMIN" (highlighted in red). The login form includes a "Username" field with a user icon, a "Password" field with a lock icon and a toggle eye icon, and a red "AUTHENTICATE ACCESS →" button. A tooltip "Please fill out this fie" points to the Username field. At the bottom, a status bar indicates "POSTGRESQL ROLE-BASED AUTHORIZATION ACTIVE".

Admin Login System

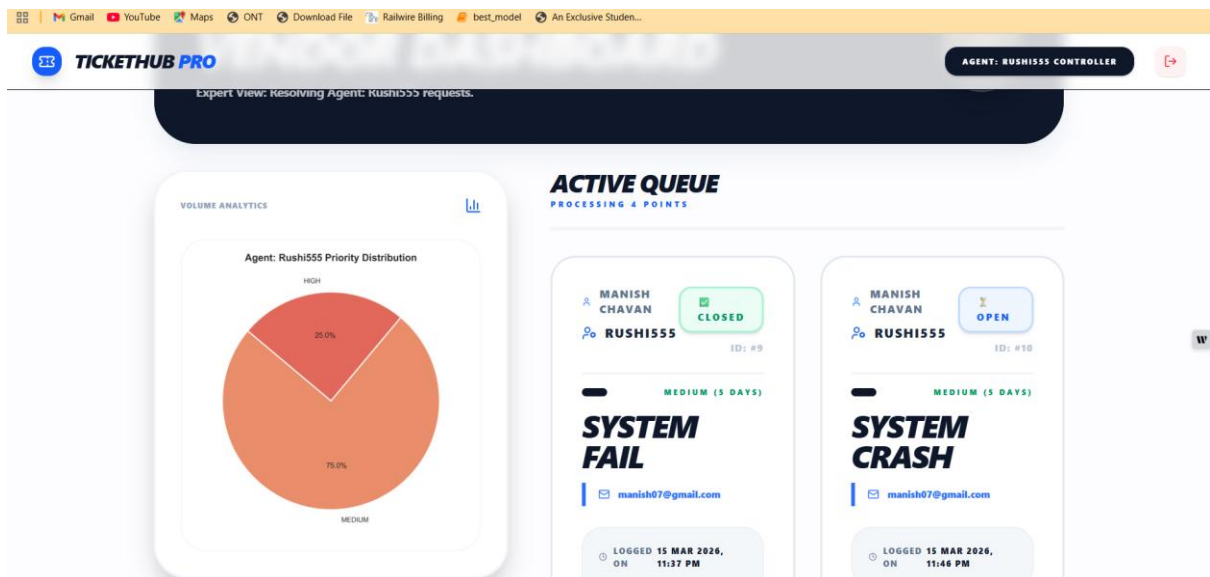


The Buyer Ticket Form UI is part of the "TICKETHUB PRO" application. It features a blue header with a shield icon and the text "DISPATCH REQUEST ENTERPRISE MASTER ENGINE V3.0 // SECURE LINK". The form includes a "TICKET SUBJECT" field, a "DOCUMENTATION" field, and a "LOCKED IDENTITY" section showing "Manish Chavan" and "manish07@gmail.com". There are two dropdown menus for "SLA PRIORITY" (set to "LOW") and "TARGET TEAM" (set to "TECHNICAL"). Below these are buttons for "ATTACH PROOF" and "DEPLOY REQUEST". The top navigation bar includes "NEW", "HISTORY", "CONTROLLER", and a red arrow icon.

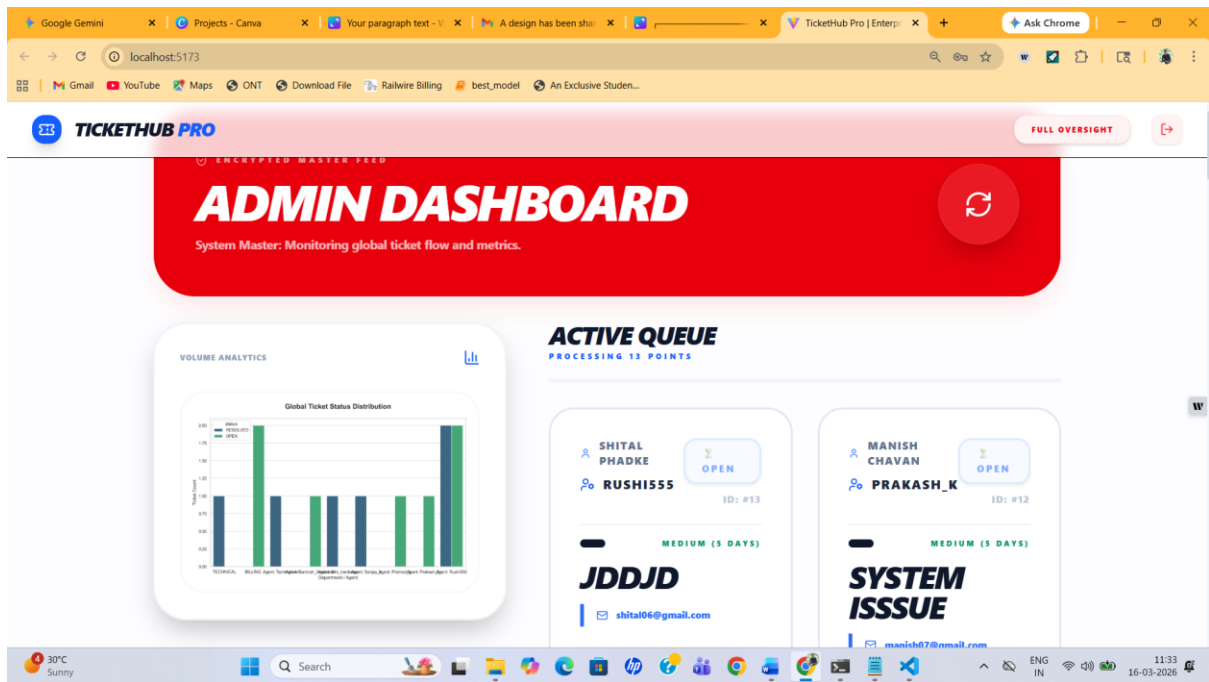
Buyer Ticket Form



Buyer Ticket History



Vendor Tickets Opened or Resolved



Admin Dashboard

9. Conclusion

The TicketHub Pro system represents a robust, enterprise-grade solution for technical incident management. By integrating a high-performance Django REST Framework backend with a massive, high-contrast React frontend, the platform bridges the gap between complex technical data and intuitive user experience.